

Foundations of Artificial Intelligence - 2026.1

Felix Mohr

Universidad de La Sabana

Chía, January 28, 2026

Course Outline

1. Agent Architecture
2. Decision Making Situations and Rational Acting
3. Generic Decision Optimization under Certainty
 - Optimization Problems
 - Constructive Search

Course Outline

1. Agent Architecture
2. Decision Making Situations and Rational Acting
3. Generic Decision Optimization under Certainty

Agents

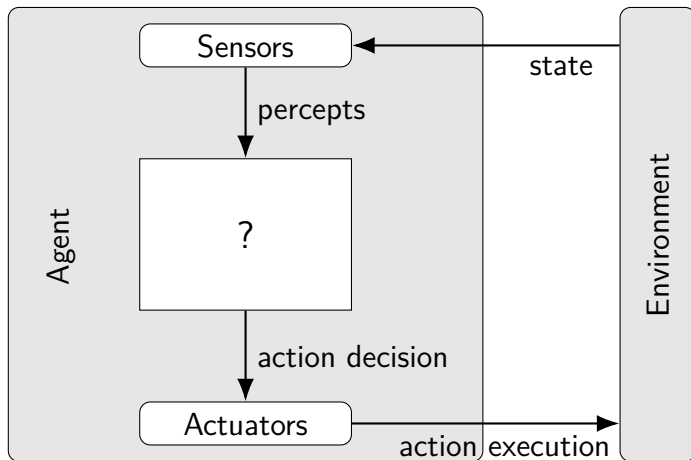
An **agent** is anything that can perceive (parts of) the state of an environment and influence events in it.

To do this, agents have

- **sensors** to capture **percepts** of a space P of its environment, and
- **actuators** to conduct **actions** of a space A in its environment.

Agents make **autonomous decisions** about which actions to perform based on the **history of percepts** (no intelligence without deliberation).

Agents and Environments



Areas of Intelligence in the Agent Program

According to the 3 components in the agent, there are 3 areas of intelligence:

- Percept Level: How I *interpret* raw signals received from sensors.
- Decision Level: How I *want* my interaction with the world to look like.
- Action Level: How I *implement* my interaction.



Agents

Agent Program

The factual realization of the agent decision making is the **agent program**.

The agent program only takes the **current percept** as an input, because there is nothing outside the agent that is able to memorize the history.

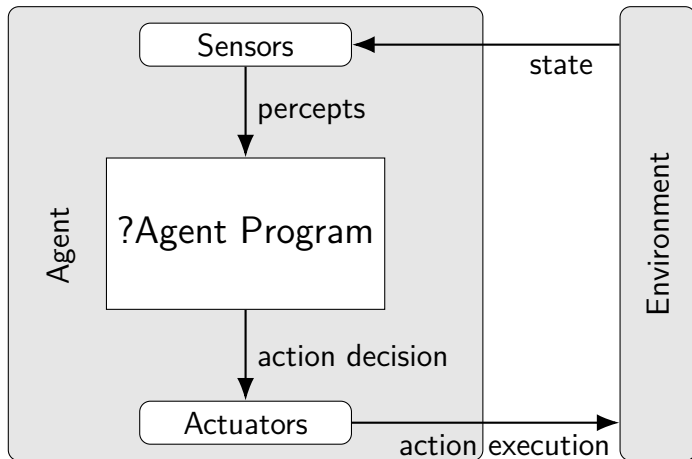
So the agent program is an *implementation* of a function

$$f : P \rightarrow A$$

and its outputs can be implemented through actuators in the environment.

Agents

The Agent Program



Agents

Properties of Agents

The agent setup of an environment also has several properties:

- Number of agents (one or many)
- For multiple agents: competitive/cooperative/teams
- Knowledge about environment physics (known vs unknown)
- Ability to observe the environment (fully vs partially)
- Ability to enforce actions (yes or no)

Agents

Examples for Properties

- Order a Pizza: non-deterministic, static, discrete, atomic/episodic, single-agent, environment unknown but observable
- Chess/Go: deterministic, static, discrete, sequential, multi-agent, environment known and observable
- Medical Treatment: non-deterministic, dynamic, continuous, atomic, single-agent, environment semi-known but only partially observable
- Autonomous Driving: non-deterministic, dynamic, continuous, sequential, multi-agent, environment known but only partially observable

Agents

Our Scope

The interaction between agents and environment can be very complex.

In this course, we will make the following simplifications:

- no sensing is necessary (this is covered in computer vision, sensor networks, etc.)
- no actuation is necessary (this is covered in robotics)

Agents

Our Scope

In this simplified scenario, the the environment loop can be implemented as follows:

- take current state s_t and use it as a percept
- compile action vector $\mathbf{a}_t := (a_t^1, a_t^2, \dots, a_t^k)$, where
 - ▶ k is the number of agents and
 - ▶ a_t^i is the i -th agent's picked action for state s_t
- use $\mathbf{a}_t$ as the event and update environment via $\psi(s_t, \mathbf{a}_t)$

So there are no effects that influence the environment other than the agents' actions.

If only one agent plays, use a dummy action for the others.

Rational and Intelligent Agents

Up to now, we have not defined any objectives:

Agents can just move inside the environment forever without any purpose or goals.

In reality, some behaviors of agents are better (maybe more intelligent) than others.

How can we say whether an agent is intelligent?

Course Outline

1. Agent Architecture
2. Decision Making Situations and Rational Acting
3. Generic Decision Optimization under Certainty

Decision Making Situations and Rational Acting

Rationality

At different points of time, agents will have the opportunity to make a decision.

An intelligent agent should take a decision that is somehow “good” for him; we would then call the agent to be **rational**.

Let A be the set of possible actions in that moment and A with a pre-order $\succeq$, where $\succeq$ encodes “appears better”. Then the task is to find a maximizer of A wrt $\succeq$.

In a simplified case, we seek an $\arg \max_a f$ for a real-valued function $f : A \rightarrow \mathbb{R}$ and

$$a \succeq a' \Leftrightarrow f(a) \geq f(a')$$

Decision Making Situations and Rational Acting

Factors That Influence Rationality

What is rational and hence should influence the definition of $\succeq$ or f depends on

- a **performance measure** that defines the criterion of success in the future world towards which the action a paves the way
- the agent's **knowledge** of the environment and actions that other agents will choose
- past **observations** of effects of own actions and other agents' decisions

An agent is then rational if it selects an action that maximizes the expected performance given its knowledge and the available actions.

Here we implicitly assumed that $\succeq$ and f are only defined over actions that are possible for the agent (not theoretically possible actions).

Decision Making Situations and Rational Acting

Atomic vs Sequential Environment

How one makes decisions depends on whether the environment is atomic or sequential:

- Atomic environments are simpler: Just one decision to be made.

The ordering $\succeq$ on actions is only affected by the immediate outcome of the action.

- Sequential environments are more complex:

The ordering $\succeq$ on actions requires to look ahead and check the implications of the next action; myopic behavior is rarely rational.

Decision Making Situations and Rational Acting

Certainty of Decision Effects

Orthogonally, agents can have different insights into the effects of their actions:

- Decisions under **certainty**: Deterministic environments where agents can fully observe the current state and have full knowledge (and control) about transitions.
- Decisions under **risk**: Non-deterministic environments where agents can fully observe the current state and have full knowledge about transitions.
- Decisions under **uncertainty**: Environments in which agents lack ability to observe the current state, don't know the transition mechanics, or both.

Decision Making Situations and Rational Acting

Decision Situation Matrix

		Action Effects		
Horizon		Certain	Risk	Uncertain
	atomic			
	sequential			

Decision Making Situations and Rational Acting

Decision Objects

To account for future decision situations subsequent to the current one more explicitly, we generally can think of some **decision object space** $\mathcal{X}$ with a pre-order $\succeq$.

The nature of objects in $\mathcal{X}$ depends on the type of the decision situation:

- a single action (atomic environment)
- an action sequence (sequential decisions under certainty)
- a policy (sequential decisions under risk or uncertainty)

We assume that we can *extract* the action for the current situation from such an object.

Decision Making Situations and Rational Acting

Challenges to Address

Let $\mathcal{X}$ be a decision object space and $g : \mathcal{X} \rightarrow \mathbb{R}$ a decision score function.

We require that any action that is extracted from a maximizer of g is a rational choice.

The rest of the course will seek to work towards these objectives:

- ➊ How to find $\arg \max_{x \in \mathcal{X}} g(x)$?
- ➋ How to even only *compute* a value $g(x)$ if g is difficult?
- ➌ How to *estimate* $g(x)$ if the agent's knowledge is limited?

Rational and Intelligent Agents

Main Building Blocks for an Agent Program that Achieves Rational Behavior

The main ingredients to build the agent program upon are:

- **knowledge**
- **inference** (deriving “new” knowledge from existing knowledge)
- **learning** (updates of the knowledge based on percepts)

Learning is needed to make agents independent of hard-wired knowledge.

Typical procedure: The agent program adopts learning to incorporate the percepts into the knowledge base. The new action is then directly dictated by the knowledge or is inferred from it via inference.

To a certain degree, these building blocks are substitutable by each other.

Rational and Intelligent Agents

Intelligent Agents

Until two decades ago, rationality was all we cared about.

Many agents were not called intelligent even if they acted rationally, because ultimately all behavior was pre-determined by the developers.

True intelligence (hard to grasp) requires some kind of autonomous learning.

An **intelligent agent** is one in which learning has enabled significant *generalized knowledge* not explicitly encoded that can be used for inference of rational decisions.

Rational and Intelligent Agents

Lee Sedol vs. AlphaGo (2016) - Intelligent Agent by learned behavior



Course Outline

1. Agent Architecture
2. Decision Making Situations and Rational Acting
3. Generic Decision Optimization under Certainty
 - Optimization Problems
 - Constructive Search

Generic Decision Optimization under Certainty

In the case of certainty, the agent has total control about the environment.

That means that he already knows at planning time that he take one (atomic) or several (sequential) actions and what the exact consequences of those actions will be.

This means that we can even forget about percepts and action execution and only need to know the initial state s_0 .

We can then assume that the computation of $g : \mathcal{X} \rightarrow \mathbb{R}$ is trivial and the “only” problem is to find one of its maximizers.

Course Outline

1. Agent Architecture
2. Decision Making Situations and Rational Acting
3. Generic Decision Optimization under Certainty
 - Optimization Problems
 - Constructive Search

Generic Decision Optimization under Certainty

Optimization Problems

An **optimization problem instance** is described by a tuple $\langle \mathcal{X}, \succeq \rangle$, where

- $\mathcal{X}$ is some set called the **search space**, e.g., agents' decisions or mental state configs
- $\succeq \subset \mathcal{X} \times \mathcal{X}$ is a pre-order on $\mathcal{X}$, and $x \succeq x'$ means that x is not worse than x'

The task associated with an optimization problem instance $\langle \mathcal{X}, \succeq \rangle$ is to find a $\max_{\succeq} \mathcal{X}$.

Generic Decision Optimization under Certainty

Optimization Problems - Remarks

Some remarks are due:

- In many algorithms, it is more efficient to use $\succ$ instead of $\succeq$, because we want to check whether a new solution is *strictly* better.
- In some cases, all elements of $\mathcal{X}$ are equally good, and the task is to just to find *any* element from $\mathcal{X}$ (**constraint satisfaction**, degenerate optimization).

Example: Solve a 10^6 tile puzzle. Here, $|\mathcal{X}| = 1$, but it is difficult to find it.

- We assume that $\mathcal{X}$ is too large to look at all its members; typically $|\mathcal{X}| \gg 10^{10}$.

Generic Decision Optimization under Certainty

Search Workspaces

Generally we might not want to work in $\mathcal{X}$ directly but some other **search workspace** $\mathcal{W}$.

For example, because we want to work with *partial* or *invalid* items from $\mathcal{X}$ or want to use a special *encoding* for efficiency reasons.

Formally, we need

- a partial and ideally bijective **recovery function** $\varphi : \mathcal{W} \rightarrow \mathcal{X}$
- a **goal predicate** $\star : \mathcal{W} \rightarrow \{0, 1\}$, where $\star(W) = 1$ iff $\varphi(W)$ is defined
- $\succeq$ is induced in $\mathcal{W}$ by its ordering in $\mathcal{X}$ and φ : $W \succeq_{\mathcal{W}} W' \Leftrightarrow \varphi(W) \succeq_{\mathcal{X}} \varphi(W')$

Generic Decision Optimization under Certainty

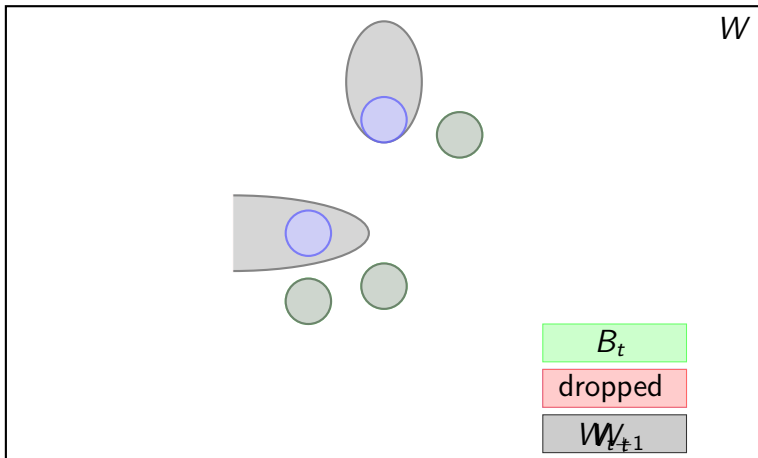
Generic Optimization Routine

All optimization algorithms can be expressed through this general scheme:

- 1: $t \leftarrow 0$
- 2: Initialize set $\mathcal{W}_0 \subseteq \mathcal{W}$ of initial work items
- 3: **while** stopping criterion is not met **do**
- 4: Choose work items $A_t \subseteq \mathcal{W}_t$ to concentrate on
- 5: Derive new work items B_t from A_t and set $\mathcal{W}_{t+1} \leftarrow \mathcal{W}_t \cup B_t$
- 6: Remove some undesired work items from $\mathcal{W}_{t+1}$
- 7: $t \leftarrow t + 1$
- 8: **end while**
- 9: **return** $\varphi(\max_{\succeq} \{W \mid W \in \mathcal{W}_t \wedge \star(W) = 1\})$

Generic Decision Optimization under Certainty

Generic Optimization Routine - Iteration Example



Generic Decision Optimization under Certainty

Two Classes of Optimization Methods

There are two major sub-patterns to implement the optimization routine:

- ➊ **Constructive Search:** Items in $\mathcal{W}$ are (partial) construction plans for items in $\mathcal{X}$ and are extended step by step, typically one $W \in \mathcal{W}$ per iteration.
 - ▶ Step-by-Step construction of a purchase list
 - ▶ Plan a truck route from scratch
- ➋ **Transformative Search:** $\mathcal{W}$ is initialized with some items from $\mathcal{X}$, and each iteration selects and manipulates some items from $\mathcal{W}$.
 - ▶ Change the items or quantities in a purchase list
 - ▶ Modify an existing truck route

Course Outline

1. Agent Architecture
2. Decision Making Situations and Rational Acting
3. Generic Decision Optimization under Certainty
 - Optimization Problems
 - Constructive Search

Generic Decision Optimization under Certainty

Constructive Search

Suppose that we can have a finite set O of operations and the workspace is $\mathcal{W} \subseteq O^*$.

Workspace items are (partial) construction plans from which we can recover items in $\mathcal{X}$.

Examples:

- Every $o \in O$ defines the quantity to be bought for a specific product.

Then every $W \in \mathcal{W}$ is a purchase list.

- Every $o \in O$ defines a destination that should be visited by a truck.

Then every $W \in \mathcal{W}$ is a tour.

Generic Decision Optimization under Certainty

General Constructive Search

A function $e : \mathcal{W} \rightarrow 2^O$ is an **expansion function** if $o \in e(W) \Leftrightarrow (W_1, \dots, W_k, o) \in \mathcal{W}$.

Independently of the precise structure of $\mathcal{W}$, we can always formulate a constructive search algorithm by pointwise extensions of the plan.

- 1 Start with $\mathcal{W}_0$ containing only an empty construction plan
- 2 In a loop, select a promising plan $W \in \mathcal{W}_t$, determine operations $expand(W)$ with which W can be extended, and add them in $\mathcal{W}_{t+1}$.

This enumerates all construction plans in $\mathcal{W}$, and we can return the best found object, or the first valid object if no optimization is required.

Generic Decision Optimization under Certainty

General Constructive Search

Require: $(\langle \text{expand}, \star, \succ \rangle)$

```
1:  $OPEN \leftarrow \{()\}$ ,  $W^* = \perp$ 
2: while  $OPEN \neq \emptyset$  and there is still time do
3:    $W \leftarrow \text{Remove-First}(OPEN)$ 
4:   for  $o \in \text{expand}(W)$  do
5:      $W' \leftarrow (W_1, \dots, W_{|W|}, o)$ 
6:     if  $\star(W')$  then
7:       if  $\succ$  is None then
8:         return  $W'$ 
9:       else if  $(W^* = \perp) \vee (W' \succ W^*)$  then
10:         $W^* \leftarrow W'$ 
11:      end if
12:    else
13:       $OPEN \leftarrow OPEN \cup \{W'\}$ 
14:    end if
15:  end for
16: end while
17: return  $W^*$ .
```

Generic Decision Optimization under Certainty

Ordering *OPEN* for General Constructive Search

OPEN organizes the workspace items in form of a **priority queue** (not FIFO). We can define any ordering of nodes in order to steer the search algorithm.

Simple prioritization can be obtained through two common data structures:

- realizing *OPEN* as a *queue* yields a **Breadth First Search (BFS)**
- realizing *OPEN* as a *stack* yields a **Depth First Search (DFS)**

Much more powerful ordering schemes exist though.

Constructive Search

Pros and Cons of Constructive Search

Advantages of constructive search include:

- It is a systematic search, and there is a (theoretic) possibility of proving optimality.
- A constructive approach may allow for large area exploration strategies.
- Can be easily used for both atomic and sequential situations.

Disadvantages of constructive search include:

- Guiding the search into promising regions can be difficult and time consuming.
- Potentially large memory consumption through *OPEN* and overhead structures.
- Possibly slow construction of solutions, specifically if objects in $\mathcal{X}$ are complex.

Summary

The whole class in three sentences:

- We assume simplified agents whose decisions determine the events for the environment.
- We want to determine rational behavior for agents, which generally requires to think ahead and account for uncertainty.
- For decision making under certainty, we mainly need to optimize the known objective function, which can be done in a constructive or transformative manner.